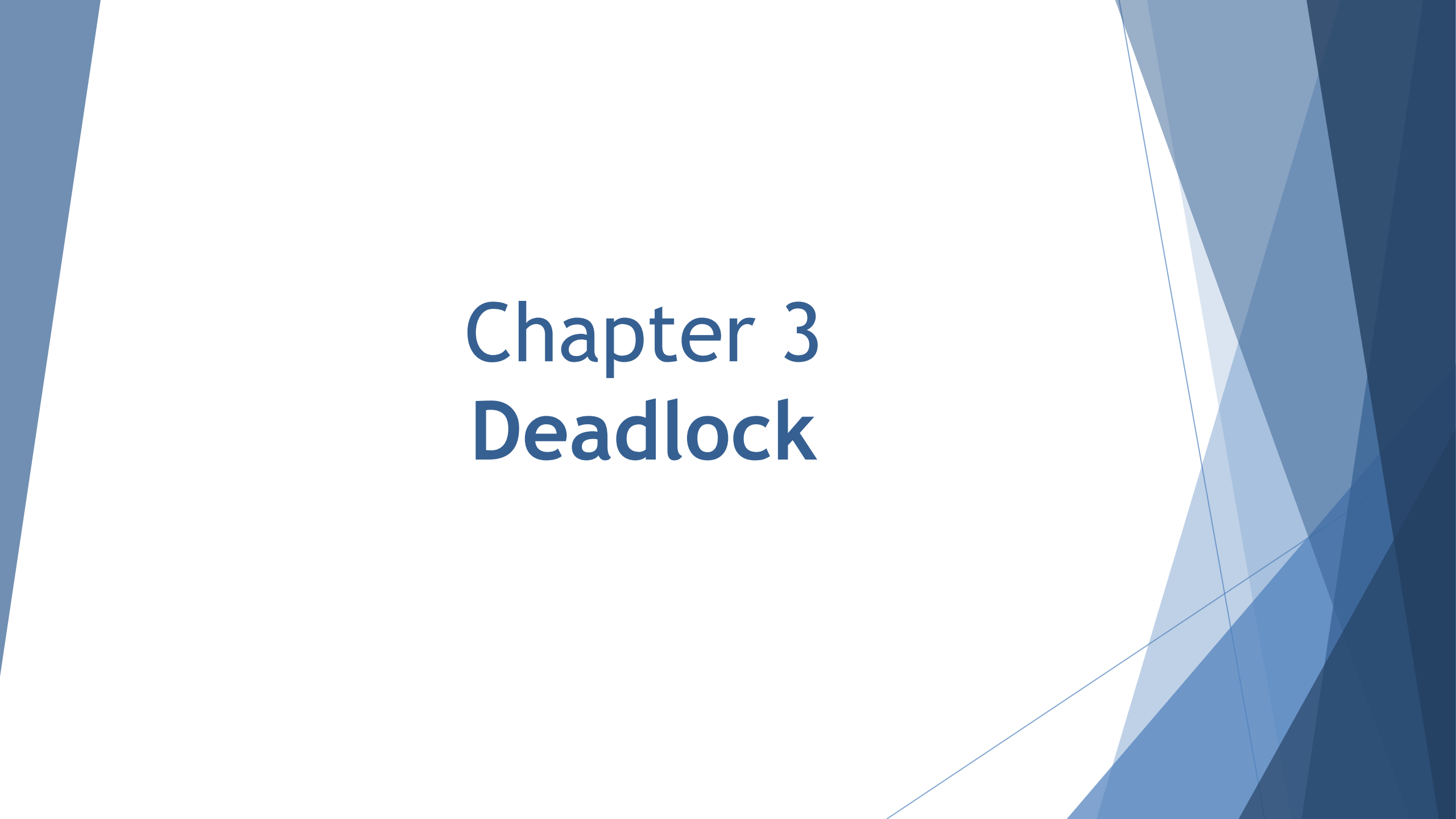


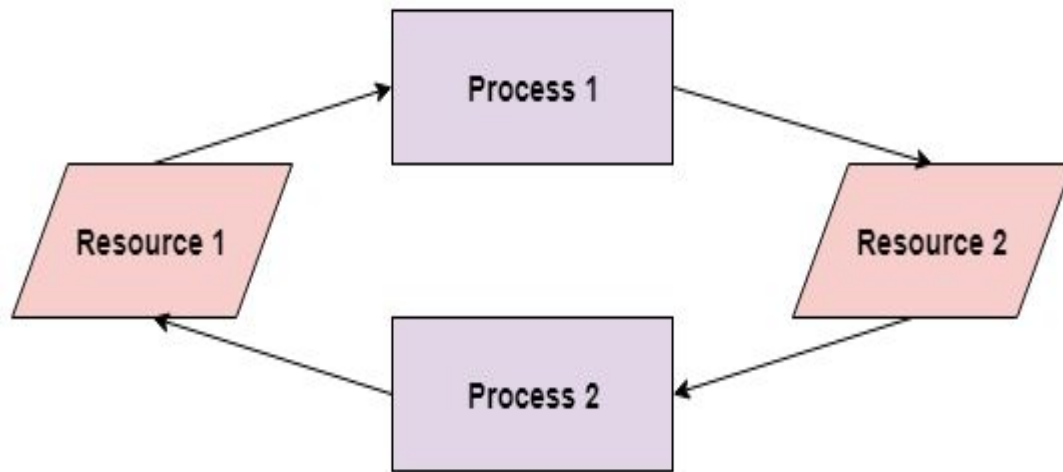


Chapter 3

Deadlock

What is Deadlock?

- ▶ **Deadlock** is a situation that occurs in OS when any process enters a waiting state because another waiting process is holding the demanded resource.
- ▶ A deadlock happens in operating system when two or more processes need some resource to complete their execution that is held by the other process.

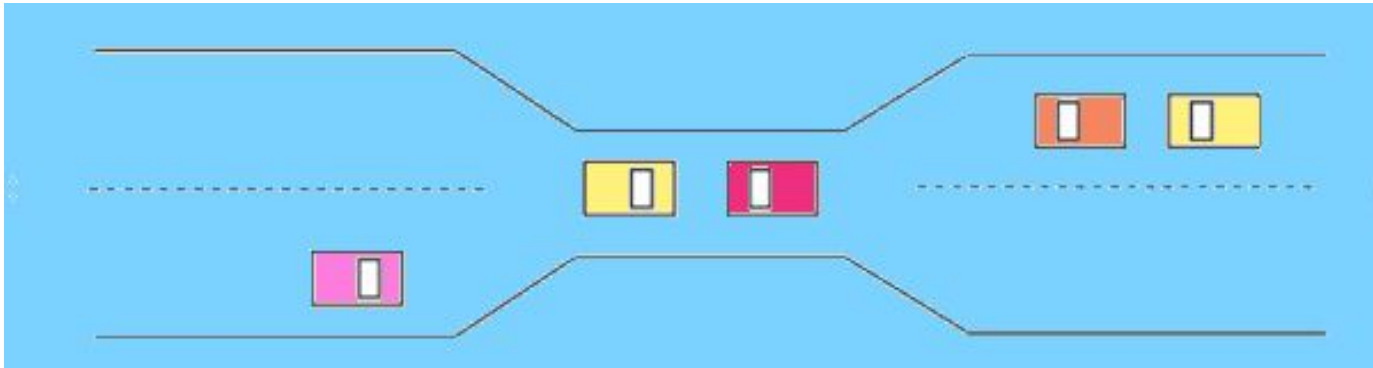


Deadlock in Operating System

- In the diagram, the process 1 has resource 1 and needs to acquire resource 2.
- Similarly process 2 has resource 2 and needs to acquire resource 1.
- Process 1 and process 2 are in deadlock as each of them needs the other's resource to complete their execution but neither of them is willing to relinquish their resources.

Example of Deadlock

- A real-world example would be traffic.
- Here, a bridge is considered a resource.
- So, when Deadlock happens, it can be easily resolved if one car backs up (Preempt resources and rollback).
- Several cars may have to be backed up if a deadlock situation occurs.



System Model

- ▶ System consists of resources
- ▶ Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- ▶ Each resource type R_i has W_i instances.
- ▶ Each process utilizes a resource as follows:
 - ▶ request
 - ▶ use
 - ▶ release

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- ▶ **Mutual exclusion:** only one process at a time can use a resource
- ▶ **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- ▶ **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- ▶ **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

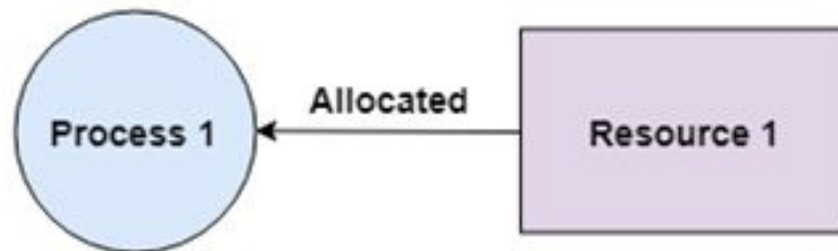
Necessary Conditions

There are four conditions that are necessary to achieve deadlock.

- ▶ Mutual Exclusion
- ▶ Hold and Wait
- ▶ No Preemption
- ▶ Circular Wait

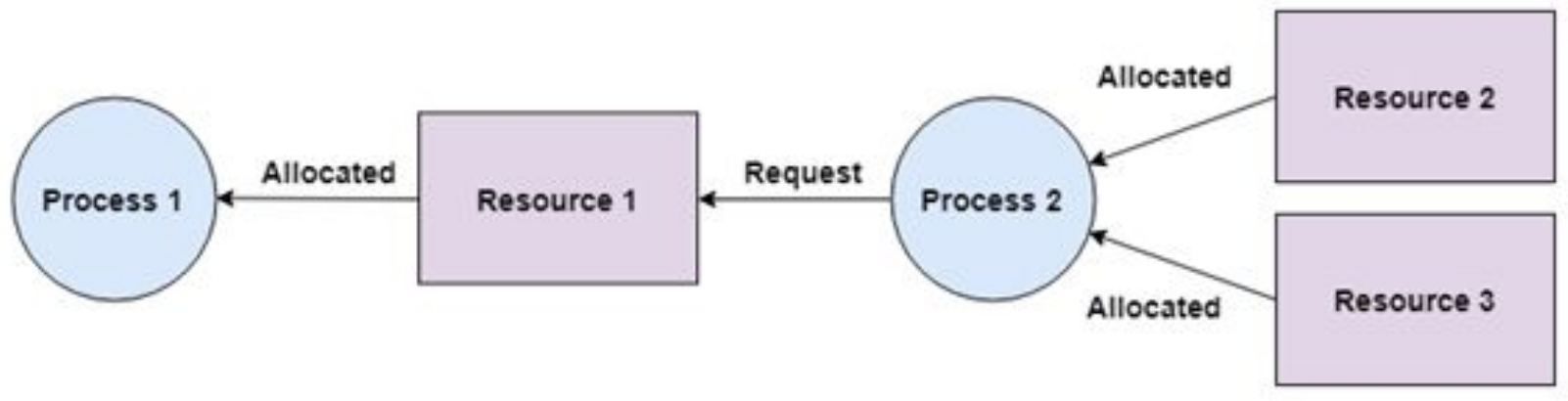
Mutual Exclusion [non sharable mode]

- ▶ There should be a resource that can only be held by one process at a time.
- ▶ If any other process requests this resource, then that process must wait for the resource to be released.
- ▶ In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



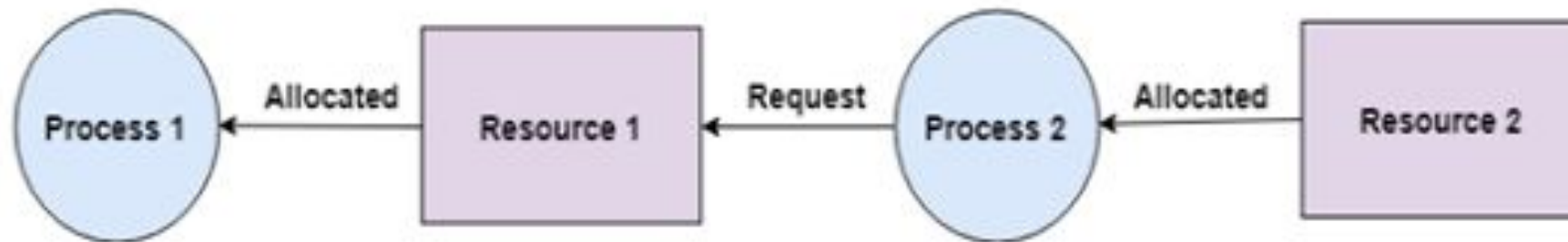
Hold and Wait

- ▶ A process can hold multiple resources and still request more resources from other processes which are holding them.
- ▶ In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



No Preemption

- ▶ A resource cannot be preempted from a process by force.
- ▶ A process can only release a resource voluntarily.
- ▶ Once a process is holding a resource then that resource cannot be taken away from that process until the process voluntarily releases it.
- ▶ In the diagram below, Process 2 cannot preempt Resource 1 from Process 1. It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.

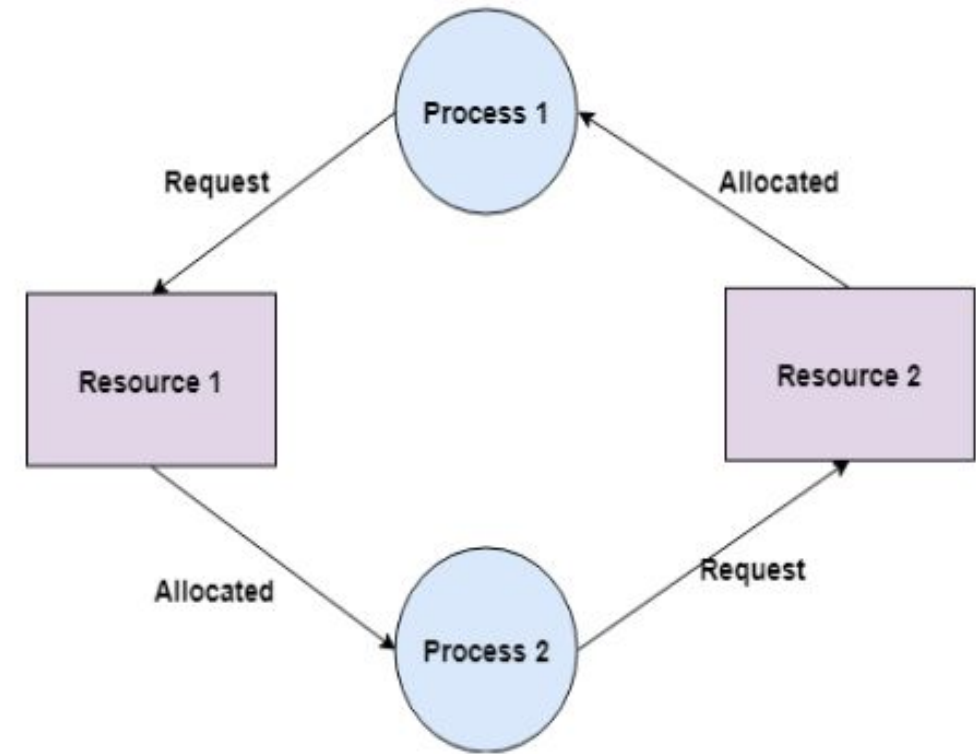


Circular Wait

- ▶ A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process.
- ▶ This forms a circular chain.

For example:

- ▶ Process 1 is allocated Resource 2 and it is requesting Resource 1. Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2.
- ▶ This forms a circular wait loop.



Methods for Handling Deadlocks

There are four ways of handling deadlocks

1. Ignore the problem all together [Ostrich method]

If deadlocks only occur once a year or so, it may be better to simply let them happen and **reboot** as necessary. Widely used by Windows and Linux.

2. Deadlock prevention

Prevent deadlocks from ever happening, by **preventing at least one** of the necessary conditions for deadlock

3. Deadlock avoidance [Bankers algorithm]

Do not allow the system to get into a deadlocked state. Check safe and unsafe condition

4. Deadlock detection and recovery

Abort a process or preempt some resources when deadlocks are detected.

Deadlock Prevention

- ▶ It is very important to prevent a deadlock before it can occur.
- ▶ So, the system checks each transaction before it is executed to make sure it does not lead to deadlock.
- ▶ If there is even a slight chance that a transaction may lead to deadlock in the future, it is never allowed to execute.

Deadlocks can be prevented by preventing at least one of the four required conditions:

- ▶ **Mutual Exclusion**
- ▶ **Hold and Wait**
- ▶ **No Preemption**
- ▶ **Circular Wait**

Deadlock Avoidance

- ▶ In some algorithms, the scheduler only needs to know the maximum number of each resource that a process might potentially use.
- ▶ When a scheduler sees that starting a process or granting resource requests may lead to future deadlocks, then that process is just not started or the request is not granted.

Banker's Algorithm (Resource-Request Algorithm) for Deadlock avoidance

- ▶ When a request is made, the scheduler determines whether granting the request would leave the system in a safe state.
- ▶ If not, then the process must wait until the request can be granted safely.

Deadlock Detection and recovery

- ▶ A deadlock can be detected by a resource scheduler as it keeps track of all the resources that are allocated to different processes.

What to do once a deadlock has occurred?

Following three strategies can be used to remove deadlock after its occurrence.

- ▶ **Preemption:**
 - ▶ We can take a resource from one process and give it to other. This will resolve the deadlock situation, but sometimes it does causes problems.
- ▶ **Rollback**
 - ▶ In situations where deadlock is a real possibility, the system can periodically make a record of the state of each process and when deadlock occurs, roll everything back to the last checkpoint, and restart, but allocating resources differently so that deadlock does not occur.
- ▶ **Kill one or more processes**
 - ▶ This is the simplest way.

Banker's Algorithm Example 1

Process	Allocation	Max Need
	A B C	A B C
P1	0 1 0	7 5 3
P2	2 0 0	3 2 2
P3	3 0 2	9 0 2
P4	2 1 1	4 2 2
P5	0 0 2	5 3 3

Total Resources available: A=10, B=5, C=7

1. Deadlock will occur or not?
2. What is the safe sequence of processes to be executed?

Remaining Need [i] = Max need [i] - Allocation [i]

- ▶ Need for P1: $(7, 5, 3) - (0, 1, 0) = 7, 4, 3$
- ▶ Need for P2: $(3, 2, 2) - (2, 0, 0) = 1, 2, 2$
- ▶ Need for P3: $(9, 0, 2) - (3, 0, 2) = 6, 0, 0$
- ▶ Need for P4: $(4, 2, 2) - (2, 1, 1) = 2, 1, 1$
- ▶ Need for P5: $(5, 3, 3) - (0, 0, 2) = 5, 3, 1$

Hence, we created the context of remaining need matrix.

Banker's Algorithm Example 1 contd..

Total Resources available: A=10, B=5, C=7

Process	Allocation	Max Need	Current available	Remaining need
	A B C	A B C	A B C	A B C
P1	0 1 0	7 5 3	3 3 2	7 4 3
P2	2 0 0	3 2 2		1 2 2
P3	3 0 2	9 0 2		6 0 0
P4	2 1 1	4 2 2		2 1 1
P5	0 0 2	5 3 3		5 3 1
Total Allocation	7 2 5			

Current available = Total Resources available - Total allocation

For A $\square 10 - 7 = 3$

For B $\square 5 - 2 = 3$

For C $\square 7 - 5 = 2$

Banker's Algorithm Example 1 contd..

Process	Allocation	Max Need	Current available	Remaining need
	A B C	A B C	A B C	A B C
P1	0 1 0	7 5 3	3 3 2	7 4 3 ✕
P2	2 0 0	3 2 2	5 3 2 [p2 resources added]	1 2 2 ←
P3	3 0 2	9 0 2	7 4 3 [p4 resources added]	6 0 0 ✕
P4	2 1 1	4 2 2	7 4 5 [p5 resources added]	2 1 1 ←
P5	0 0 2	5 3 3	7 5 5 [p1 resources added]	5 3 1 ←
Total Allocation	7 2 5		10 5 7 [p3 resources added]	

Safe Sequence of processes: P2 P4 P5 P1 P3

Apply the Banker's Algorithm:

- ▶ Available Resources of A, B and C are 3, 3, and 2.
- ▶ Now we check if each type of resource request is available for each process.

Step 1: For Process P1:

- ▶ Need $\leq$ Available
- ▶ 7, 4, 3 $\leq$ 3, 3, 2 condition is false.
- ▶ So, we examine another process, P2.

Step 2: For Process P2:

- ▶ Need $\leq$ Available
- ▶ 1, 2, 2 $\leq$ 3, 3, 2 condition true
- ▶ New available = available + Allocation
= (3, 3, 2) + (2, 0, 0)
= 5, 3, 2

Similarly, we examine another process P3.

Step 3: For Process P3:

- ▶ P3 Need $\leq$ Available
- ▶ $6, 0, 0 \leq 5, 3, 2$ condition is false.

Similarly, we examine another process, P4.

Step 4: For Process P4:

- ▶ P4 Need $\leq$ Available
- ▶ $2, 1, 1 \leq 5, 3, 2$ condition is **true**
- ▶ New Available resource = Available + Allocation
= $5, 3, 2 + 2, 1, 1$
= $7, 4, 3$

Similarly, we examine another process P5.

Step 5: For Process P5:

- ▶ P5 Need $\leq$ Available
- ▶ 5, 3, 1 $\leq$ 7, 4, 3 condition is **true**
- ▶ New available resource = Available + Allocation
7, 4, 3 + 0, 0, 2 $\Rightarrow$ 7, 4, 5
- ▶ Now, we again examine each type of resource request for processes P1 and P3.

Step 6: For Process P1:

- ▶ P1 Need $\leq$ Available
- ▶ 7, 4, 3 $\leq$ 7, 4, 5 condition is **true**
- ▶ New Available Resource = Available + Allocation
7, 4, 5 + 0, 1, 0 $\Rightarrow$ 7, 5, 5

So, we examine another process P2.

Step 7: For Process P3:

- ▶ P3 Need $\leq$ Available
- ▶ $6, 0, 0 \leq 7, 5, 5$ condition is true
- ▶ New Available Resource = Available + Allocation
 $7, 5, 5 + 3, 0, 2 \Rightarrow 10, 5, 7$
- ▶ Hence, we execute the banker's algorithm to find the safe state and the safe sequence like **P2, P4, P5, P1 and P3.**

Bankers algorithm example 2

Process	Max	Allocation	Available
	A, B, C, D	A, B, C, D	A, B, C, D
P0	6 0 1 2	4 0 0 1	3 2 1 1
P1	2 7 5 0	1 1 0 0	
P2	2 3 5 6	1 2 5 4	
P3	1 6 5 3	0 6 3 3	
P4	1 6 5 6	0 2 1 2	

Using Banker's algorithm, answer the following questions:-

- How many resources of type A, B, C, D are there?
- What are the contents of remaining need matrix?
- Find if the system is in safe state? If it is, find the safe sequence.

Solution

1) How many resources of type A, B, C, D are there?

Current available = Total Resources available - Total allocation

So, Current available + Total allocation = Total Resources available

- ▶ For A = $4+1+1+0+0+3$ = 9
- ▶ For B = $0+1+2+6+2+2$ = 13
- ▶ For C = $0+0+5+3+1+1$ = 10
- ▶ For D = $1+0+4+3+2+1$ = 11

Solution contd..

2) What are the contents of remaining need matrix?

	A	B	C	D
P0	2	0	1	1
P1	1	6	5	0
P2	1	1	0	2
P3	1	0	2	0
P4	1	4	4	4

3) YES , system is in safe state

Safe Sequence: P0, P1, P2, P3, P4

Process	Max need	Allocation	Available	Remaining need
	A B C D	A B C D	A B C D	A B C D
P0	6 0 1 2	4 0 0 1	3 2 1 1	2 0 1 1 ←
P1	2 7 5 0	1 1 0 0	7 2 1 2	1 6 5 0 ✗
P2	2 3 5 6	1 2 5 4	8 4 6 6	1 1 0 2 ←
P3	1 6 5 3	0 6 3 3	8 10 9 9	1 0 2 0 ←
P4	1 6 5 6	0 2 1 2	8 12 10 11	1 4 4 4 ←

□ P0 will execute

Available, $3\ 2\ 1\ 1 + 4\ 0\ 0\ 1 = 7\ 2\ 1\ 2$

□ P2 will execute

Available, $7\ 2\ 1\ 2 + 1\ 2\ 5\ 4 = 8\ 4\ 6\ 6$

□ P3 will execute

Available, $8\ 4\ 6\ 6 + 0\ 6\ 3\ 3 = 8\ 10\ 9\ 9$

□ P4 will execute

Available, $8\ 10\ 9\ 9 + 0\ 2\ 1\ 2 = 8\ 12\ 10\ 11$

□ P1 will execute

Available, $8\ 12\ 10\ 11 + 1\ 1\ 0\ 0 = 9\ 13\ 10\ 11$

Safe Sequence: P0, P2, P3, P4, P1

Banker's Algorithm Example Solutions

Exercise 1

Assume that there are 5 processes, P_0 through P_4 , and 4 types of resources. At T_0 we have the following system state:

Max Instances of Resource Type A = 3 (2 allocated + 1 Available)

Max Instances of Resource Type B = 17 (12 allocated + 5 Available)

Max Instances of Resource Type C = 16 (14 allocated + 2 Available)

Max Instances of Resource Type D = 12 (12 allocated + 0 Available)

<u>Given Matrices</u>												
	<u>Allocation Matrix</u> (N0 of the allocated resources By a process)				<u>Max Matrix</u> Max resources that may be used by a process				<u>Available Matrix</u> Not Allocated Resources			
	A	B	C	D	A	B	C	D	A	B	C	D
P ₀	0	1	1	0	0	2	1	0	1	5	2	0
P ₁	1	2	3	1	1	6	5	2				
P ₂	1	3	6	5	2	3	6	6				
P ₃	0	6	3	2	0	6	5	2				
P ₄	0	0	1	4	0	6	5	6				
Total	2	12	14	12								

need₀ (0,1,0,0) <= work(1,5,2,0)

P0 will be
executed because
need(P0) <= Work
P0 will be True

Finish matrix	
P₀ - 1	True
P ₁	False
P ₂	False
P ₃	False
P ₄	False

P0 will release the allocated
resources(0,1,1,0)
Work = Work
(1,5,2,0)+Allocated(P0)
(0,1,1,0) = 1,6,3,0

Work vector			
1	6	3	0

Need₁ (0,4,2,1) <= work(1,6,3,0) Condition Is False P1 will Not be executed

Need₂ (1,0,0,1) <= work(1,6,3,0) Condition Is False P2 will Not be executed

Need₃ (0,0,2,0) <= work(1,6,3,0) P3 will be executed

P3 will be
executed because
need(P3) <= Work
P3 will be True

Finish matrix	
P₀ - 1	True
P1	False
P ₂	False
P₃ -2	True
P ₄	False

P3 will release the allocated
resources (0,6,3,2)
Work = Work
(1,6,3,0)+Allocated(P3)
(0,6,3,2) = 1,12,6,2

Work vector			
1	12	6	2

Need₄ (0,6,4,2) <= work(1,12,6,2) P4 will be executed

P4 will be
executed because
need(P4) <= Work
P4 will be True

Finish matrix	
P₀ - 1	True
P1	False
P ₂	False
P₃ -2	True
P₄ -3	True

P4 will release the allocated
resources (0,0,1,4)
Work = Work
(1,12,6,2) + Allocated(P4)
(0,0,1,4) = 1,12,7,6

Work vector			
1	12	7	6

Need₁ (0,4,2,1) <= work(1,12,7,6) P1 will be executed

P1 will be
executed because
need(P1) <= Work
P1 will be True

Finish matrix	
P ₀ - 1	True
P ₁ -4	True
P ₂	False
P ₃ -2	True
P ₄ -3	True

P1 will release the allocated
resources (1,2,3,1)
Work = Work
(1,12,7,6) + Allocated(P1)
(1,2,3,1) = 2,14,10,7

Work vector			
2	14	10	7

Need₂ (1,0,0,1) <= work(2,14,10,7) P2 will be executed

P2 will be
executed because
need(P2) <= Work
P2 will be True

Finish matrix	
P ₀ - 1	True
P ₁ -4	True
P ₂ -5	True
P ₃ -2	True
P ₄ -3	True

P2 will release the allocated
resources (1,3,6,5)
Work = Work
(2,14,10,7) + Allocated(P1)
(1,3,6,5) = 3,17,16,12

Work vector			
3	17	16	12

The system is in a safe state and the processes will be executed in the following order:

P0,P3,P4,P1,P2

Exercise 2:

If the system is in a safe state, can the following requests be granted, why or why not? Please also run the safety algorithm on each request as necessary.

- a. P1 requests (2,1,1,0)

We cannot grant this request, because we do not have enough available instances of resource A.

- b. P1 requests (0,2,1,0)

There are enough available instances of the requested resources, so first let's pretend to accommodate the request and see the system looks like:

	Allocation				Max				Available				Need Matrix				
	A	B	C	D	A	B	C	D	A	B	C	D		A	B	C	D
P ₀	0	1	1	0	0	2	1	0	1	3	1	0	P ₀	0	1	0	0
P ₁	1	4	4	1	1	6	5	2					P ₁	0	2	1	1
P ₂	1	3	6	5	2	3	6	6					P ₂	1	0	0	1
P ₃	0	6	3	2	0	6	5	2					P ₃	0	0	2	0
P ₄	0	0	1	4	0	6	5	6					P ₄	0	6	4	2

Need₁ (0,2,1,1) is not less than work, so we need to move on to P₂.

Need₂ (1,0,0,1) is not less than work, so we need to move on to P₃.

Need₃ (0,0,2,0) is less than or equal to work.
Let's update work and finish:

Work vector	Finish matrix	
1	P ₀	True
10	P ₁	False
5	P ₂	False
2	P ₃	True
	P ₄	False

Let's take a look at Need₄ (0,6,4,2). This is less than work, so we can update work and finish:

Work vector	Finish matrix	
1	P ₀	True
10	P ₁	False
6	P ₂	False
6	P ₃	True
	P ₄	True

We can now go back to P₁. Need₁ (0,2,1,1) is less than work, so work and finish can be updated:

Work vector	Finish matrix	
1	P ₀	True
14	P ₁	True
10	P ₂	False
7	P ₃	True
	P ₄	True

Finally, Need₂ (1,0,0,1) is less than work, so we can also accommodate this. Thus, the system is in a safe state when the processes are run in the following order:

P₀, P₃, P₄, P₁, P₂. We therefore can grant the resource request.

Exercise 3

Assume that there are three resources, A, B, and C. There are 4 processes P_0 to P_3 . At T_0 we have the following snapshot of the system:

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P ₀	1	0	1	2	1	1	2	1	1
P ₁	2	1	2	5	4	4			
P ₂	3	0	0	3	1	1			
P ₃	1	0	1	1	1	1			

1.Create the need matrix.

	Need		
	A	B	C
P ₀	1	1	0
P ₁	3	3	2
P ₂	0	1	1
P ₃	0	1	0

2. Is the system in a safe state? Why or why not?

Need₁ (3,3,2) is not less than work, so we have to move on to P_2 .

Need ₂ (0,1,1) is less than work, let's update work and finish:			Need ₃ (0,1,0) is less than work, we can update work and finish:		
Work vector		Finish matrix		Work vector	
6		P_0	True	7	
1		P_1	False	1	
2		P_2	True	3	
		P_3	False		

We now need to go back to P_1 . Need₁ (3,3,2) is not less than work, so we cannot continue. Thus, the system is not in a safe state.

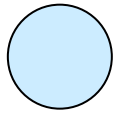
Resource Allocation Graph in Operating System

Resource-Allocation Graph

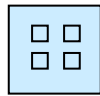
- ▶ A set of vertices V and a set of edges E .
- ▶ V is partitioned into two types:
 - ▶ $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - ▶ $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- ▶ **request edge** - directed edge $P_i \rightarrow R_j$
- ▶ **assignment edge** - directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph

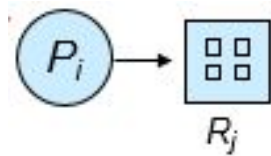
- ▶ Process



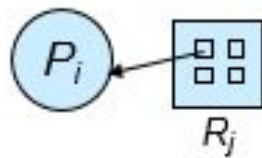
- ▶ Resource Type with 4 instances



- ▶ P_i requests instance of R_j



- ▶ P_i is holding an instance of R_j



Resource Allocation Graph in Operating System

- ▶ We use the resource allocation graph for the pictographic representation of the state of a system.
- ▶ Resource allocation graph consists of all the information which is related to all the instances of the resources means the information about available resources and the resources which the process is being using.
- ▶ In the Resource Allocation Graph, we use a circle to represent the process and rectangle to represent the resource.

Components of RAG (Resource Allocation Graph)

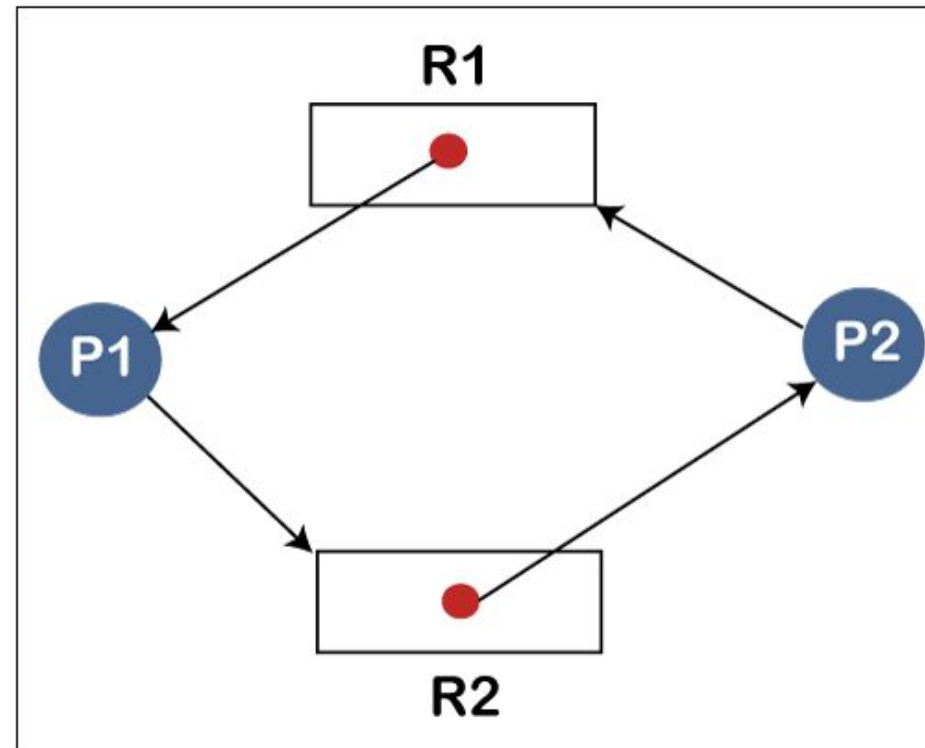
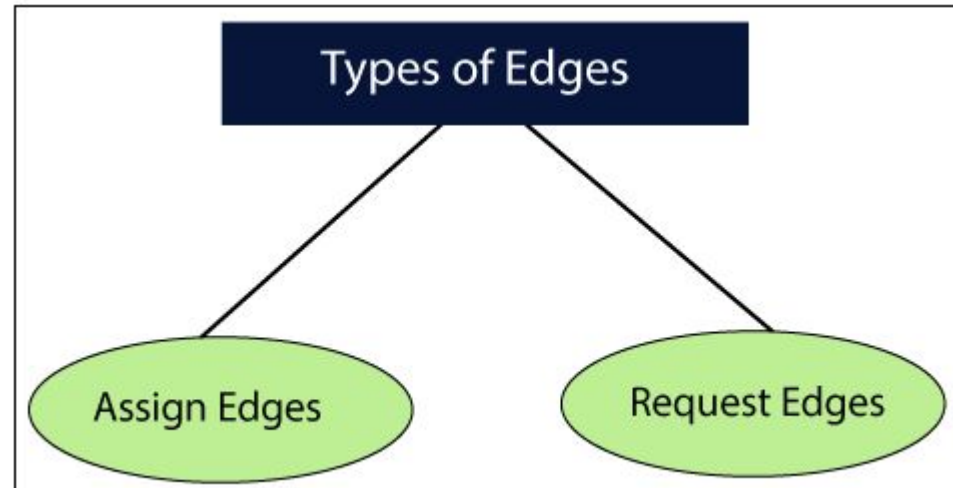
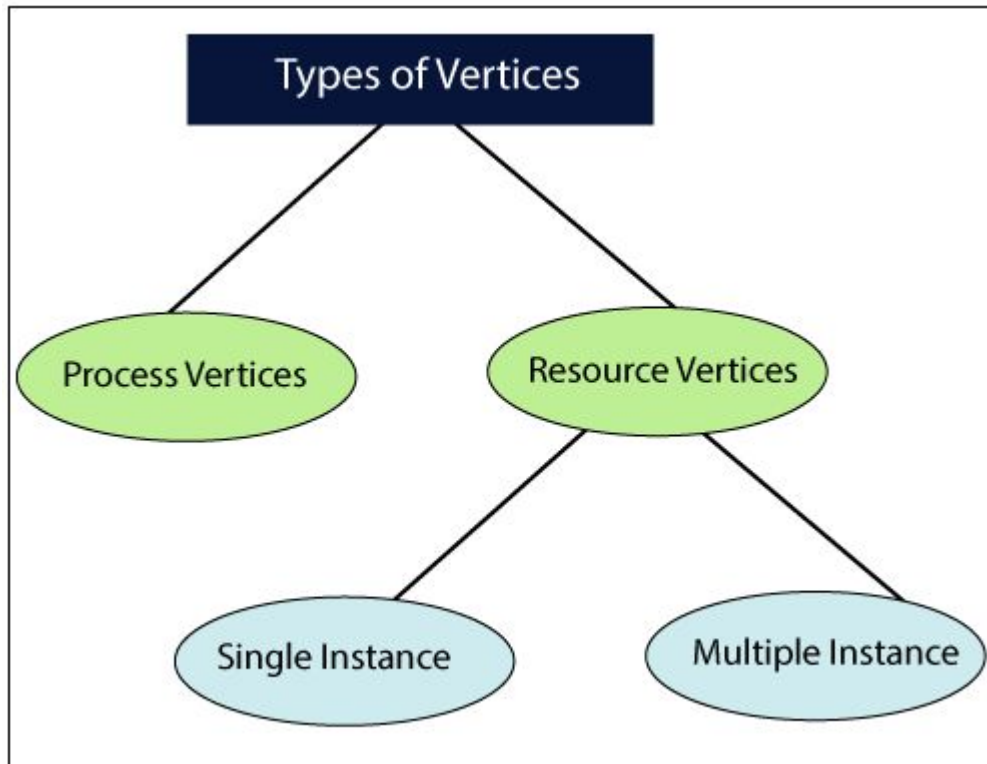
- ▶ There are two components of the resource allocation graph:
 - ▶ Vertices
 - ▶ Edges
- ▶ **Vertices:** - In the resource allocation graph, we use two kinds of vertices:
 - ▶ Process Vertices
 - ▶ Resource Vertices
- ▶ **Process Vertices:** - To represent a process, we use process vertices. We draw the process vertices by using a circle, and inside the circle, we mention the name of the process.
- ▶ **Resource Vertices:** - To represent a resource, we use resource vertices. We draw the resource vertices by using a rectangle, and we use dots inside the circle to mention the number of instances of that resource.

Resource Vertices

- ▶ According to the number of instances that may exist in the system, there are two types of resource vertices,
 - ▶ Single instance
 - ▶ Multiple instances
- ▶ **Single instance resource type:** - In single instance resource type, we use only a single dot inside the box. The single dot indicates that there is one instance of the resource.
- ▶ **Multiple instance resource type:** - In multiple instance resource type, we use multiple dots inside the box. Multiple dots indicate that there are various instances of the resources.

Edges

- ▶ There are two types of edges we use in the resource allocation graph:
 - ▶ Assign Edges
 - ▶ Request Edges
- ▶ **Assign Edges:** - We use an assign edge to represent the allocation of resources to the process. We draw assign edges with the help of arrow in which the arrow head points the process, and the process tail points the instance of the resource.
- ▶ **Request Edges:** - We use request edge to signify the waiting state of the process. Just like in assign edge, an arrow is used to draw arrow edge. Here, the arrow head points the instance of a resource, and tail of the process points to the process. For example, if a process needs 'n' instances of resource type, then we will draw 'n' assign edges.



Deadlock Detection-

Using Resource Allocation Graph, it can be easily detected whether system is in a **Deadlock** state or not.

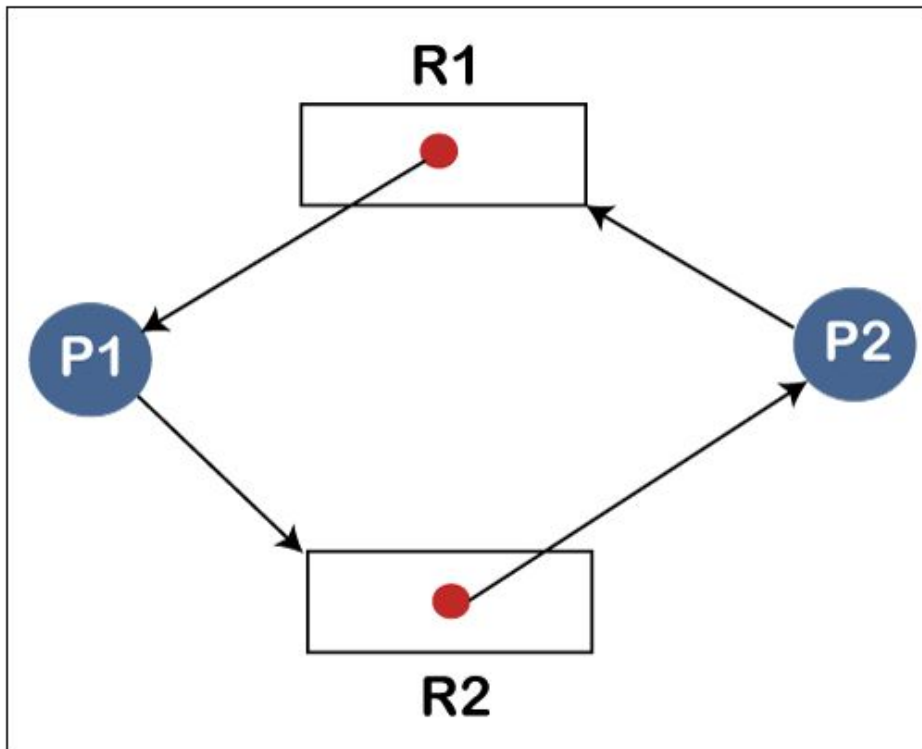
Rule-01:

- In a Resource Allocation Graph where all the resources are single instance, If a cycle is being formed, then system is in a deadlock state.
- If no cycle is being formed, then system is not in a deadlock state.

Rule-02:

- In a Resource Allocation Graph where all the resources are **NOT** single instance, If a cycle is being formed, then system may be in a deadlock state.
- **Banker's Algorithm** is applied to confirm whether system is in a deadlock state or not.
- If no cycle is being formed, then system is not in a deadlock state.
- Presence of a cycle is a necessary but not a sufficient condition for the occurrence of deadlock.

PROBLEM 1 BASED ON DETECTING DEADLOCK USING RAG [Single Instance Resource Type]



Consider the resource allocation graph in the figure.

Find if the system is in a deadlock state otherwise find a safe sequence.

Solution:

Method-01:

- ▶ The given resource allocation graph is single instance with a cycle contained in it.
- ▶ Thus, the system is definitely in a deadlock state.

Method-02:

	Allocation		Request	
	R1	R2	R1	R2
Process P1	1	0	0	1
Process P2	0	1	1	0

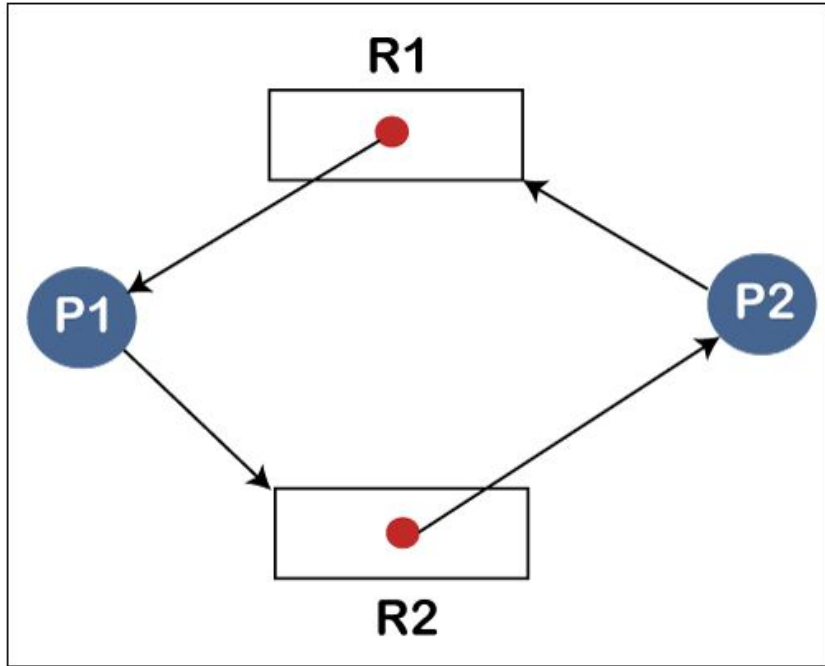
Current Available = [R1 R2] = [0 0]

Now,

- There are no instances available currently and both the processes require a resource to execute.
- Therefore, none of the process can be executed and both keeps waiting infinitely.
- Thus, the system is in a deadlock state.

Example of (RAG) Resource Allocation Graph

Example of Single Instance Resource Type



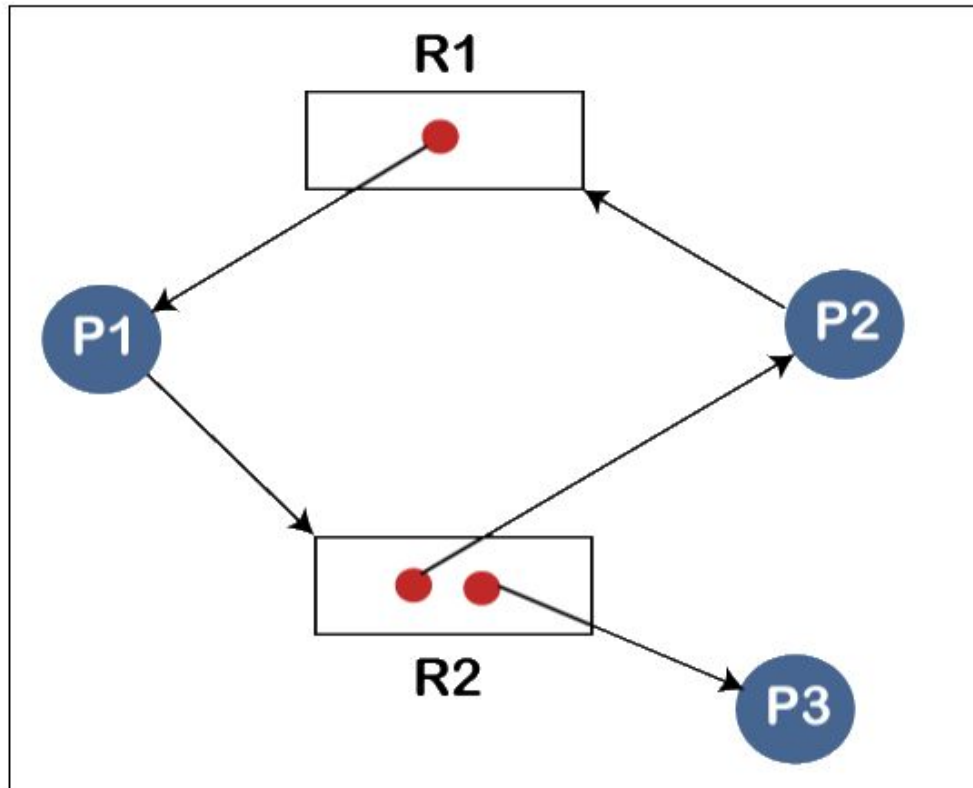
Current Available = [R1 R2] = [0 0]

In the given example, we have two processes P1, and P2 and two resources which are R1, and R2. This example is a kind of single instance resource type, and it contain a cycle, so **there is a deadlock in the system.**

	Allocation		Request	
	R1	R2	R1	R2
Process P1	1	0	0	1
Process P2	0	1	1	0

- There is no instance of resource available, and to execute the process we need a resource.
- So, no process will be executed, and both the processes keep waiting for a long time.
- We can say that there is a deadlock in the system.

PROBLEM 2 BASED ON DETECTING DEADLOCK USING RAG [Multiple Instance Resource Type]



Consider the resource allocation graph in the figure

Find if the system is in a deadlock state otherwise find a safe sequence.

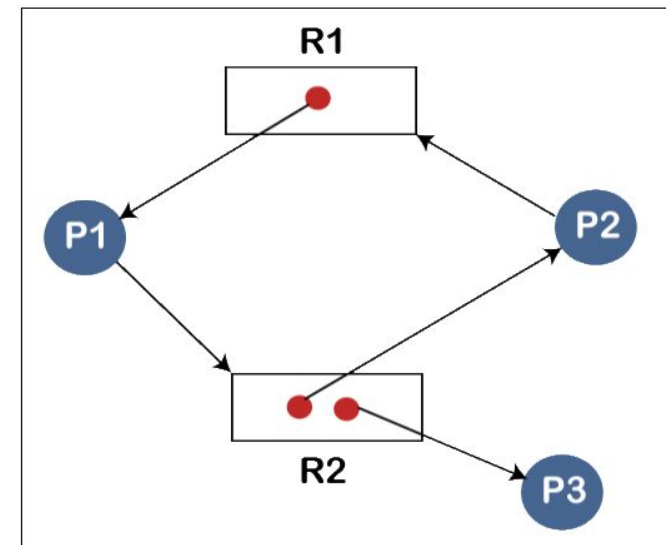
Solution-

- ▶ The given resource allocation graph is multi instance with a cycle contained in it.
- ▶ So, the system may or may not be in a deadlock state.

Now, Using the given resource allocation graph, we have-

	Allocation		Request	
	R1	R2	R1	R2
Process P1	1	0	0	1
Process P2	0	1	1	0
Process P3	0	1	0	0

current Availability = [R1 R2] = [0 0]



Solution contd..

Step-01:

Since process P3 does not need any resource, so it executes.
After execution, process P3 release its resources.

Then,

Available

$$= [0 \ 0] + [0 \ 1]$$

$$= [0 \ 1]$$

Solution contd..

Step-02:

With the instances available currently, only the requirement of the process P1 can be satisfied.

So, process P1 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [0 \ 1] + [1 \ 0]$$

$$= [1 \ 1]$$

Solution contd..

Step-03:

With the instances available currently, the requirement of the process P2 can be satisfied.

So, process P2 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [1 \ 1] + [0 \ 1]$$

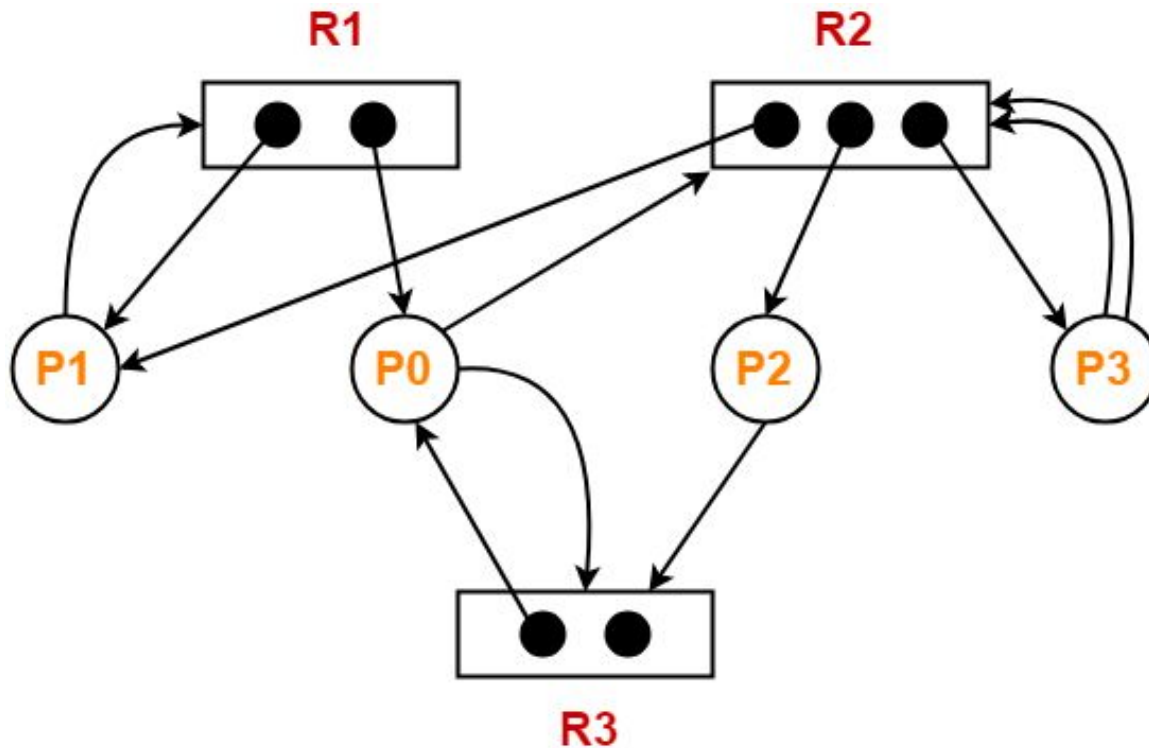
$$= [1 \ 2]$$

Solution contd..

Thus,

- ▶ There exists a safe sequence P3, P1, P2 in which all the processes can be executed.
- ▶ So, the system is in a safe state.

PROBLEM 3 BASED ON DETECTING DEADLOCK USING RAG [Multiple Instance Resource Type]



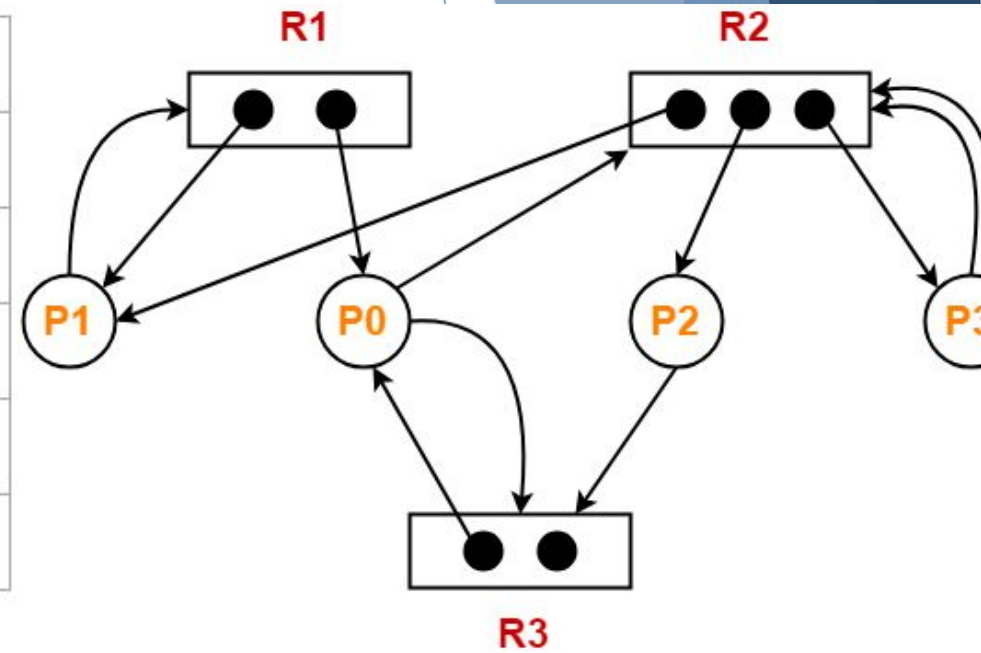
Consider the resource allocation graph in the figure

Find if the system is in a deadlock state otherwise find a safe sequence.

Solution:

	Allocation			Need		
	R1	R2	R3	R1	R2	R3
Process P0	1	0	1	0	1	1
Process P1	1	1	0	1	0	0
Process P2	0	1	0	0	0	1
Process P3	0	1	0	0	2	0

Available = [R1 R2 R3] = [0 0 1]



Solution contd..

Step-01:

With the instances available currently, only the requirement of the process P2 can be satisfied.

So, process P2 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [0 \ 0 \ 1] + [0 \ 1 \ 0]$$

$$= [0 \ 1 \ 1]$$

Solution contd..

Step-02:

With the instances available currently, only the requirement of the process P0 can be satisfied.

So, process P0 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [0 \ 1 \ 1] + [1 \ 0 \ 1]$$

$$= [1 \ 1 \ 2]$$

Solution contd..

Step-03:

With the instances available currently, only the requirement of the process P1 can be satisfied.

So, process P1 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [1 \ 1 \ 2] + [1 \ 1 \ 0]$$

$$= [2 \ 2 \ 2]$$

Solution contd..

Step-04:

With the instances available currently, the requirement of the process P3 can be satisfied.

So, process P3 is allocated the requested resources.

It completes its execution and then free up the instances of resources held by it.

Then-

Available

$$= [2 \ 2 \ 2] + [0 \ 1 \ 0]$$

$$= [2 \ 3 \ 2]$$

Solution contd..

Thus,

- ▶ There exists a **safe sequence P2, P0, P1, P3** in which all the processes can be executed.
- ▶ So, the system is in a safe state.